

EREN MENGES

New York, NY · eren.menges@nyu.edu · (929) 651-7978 · erenmenges.com · github.com/erenmenges

EDUCATION

B.S. Computer Science, Math Minor — NYU Tandon · GPA: 3.50 (4.0 last semester)

Expected May 2028

- Courses: Data Structures and Algorithms, Operating Systems, Linear Algebra, Probability/Stats, Machine Learning (Fall 2026)

SKILLS

Deep Learning: PyTorch · MLX · Quantization (QAT, PTQ, ternary/1.58-bit) · Speculative Decoding · Transformers · NumPy

Programming/Systems: Python · C++20 (threads, mutex/condvar, futures) · Git · Linux · CMake · Bash

Foundations: Backprop, CNNs, ResNet, ViT, and Transformer implementations from scratch in NumPy and PyTorch

Languages: English (Fluent), German (TELC B2), Turkish (Native)

EXPERIENCE

Machine Learning Research Intern, C2SMART Center for Urban Mobility, NYU

Jun 2025 - Aug 2025

- Trained and benchmarked 7 trajectory prediction models on Argoverse2 (0.56–0.80m error at 3.4–16ms inference).
- Showed the EMP architecture matches near state-of-the-art accuracy at ~50% fewer FLOPs with 6× faster training (8h vs 50h).

PROJECTS & OPEN SOURCE CONTRIBUTIONS

Avenue – Ternary-Native Language Models — Python · PyTorch — [code](#) · [explainer](#) · [models](#)

Jul 2026 - Aug 2026

- Built and trained six language models (20M/40M/60M, each in BF16 & 1.58-bit ternary) from scratch in 43 hours on a MacBook Pro: a custom Transformer with RoPE/RMSNORM, Muon+AdamW training pipeline, an inference stack with sampling, and a BPE tokenizer trained on the corpus.
- Ran a controlled comparison of BF16, quantization-aware training (QAT), and post-training quantization (PTQ) at three sizes. A ternary model needs ~1.9× the parameters to match BF16 quality, but at 2 bits per weight it still comes out smaller: a 33 MB ternary model beats a 41 MB BF16 model on bits-per-byte.
- Cut PyTorch/MPS training time ~40% via a RoPE rewrite, chunked linear cross-entropy, and attention-head sweeps.

Concurrent LLM Inference Server — C++20 · llama.cpp — [code](#)

Aug 2026 - Sept 2026

- Built a multi-threaded HTTP inference server in C++ that runs a local open-weight LLM (Qwen3-0.6B). A thread pool handles HTTP, a single engine thread owns the model and runs inference, and a mutex + condition_variable queue connects them. HTTP threads block on a std::future until the engine fulfills the matching std::promise.
- Scheduler uses continuous batching to minimize idle CPU capacity, and batch admission is decided on KV cache capacity.
- Load-tested to 4,096 concurrent clients with zero failed requests; ~120 tok/s steady up to 2,048 clients on a 4-core Linux VM.

MLX (Apple's ML framework for Apple silicon) — Python — [PR #4215](#) · Merged

Aug 2026

- Fixed MultiOptimizer crash on models containing parameterless modules (ReLU/GELU). Traced to tree_merge failing when merging two empty trees. Added regression tests as well. Merged in 24 hours.

RESEARCH

Should your draft model be ternary? — LLM Inference Acceleration with Speculative Decoding

Aug 2026

(sole author, under review at a NeurIPS 2026 workshop) · [paper](#) · [code](#)

- Asked whether the small draft model in speculative decoding should have ternary weights $\{-1, 0, +1\}$. On consumer hardware the draft's cost is set by its memory footprint, and ternary weights take 2 bits each instead of 16.
- Wrote everything from scratch in PyTorch, including speculative decoding. Evaluated 21 draft models (99M-1.5B) on 2 targets, 3 domains, 1,024 prompts. Real speculative decoding runs matched the predicted speedups within 0.19 tokens.
- Found that ternary drafts achieve a 2.05–2.28× predicted speedup at every size tested, while FP16 drafts fall to 1.18× at 1.5B. At equal memory, ternary drafts are accepted more often; FP16 needs up to 1.74× the memory to match a ternary draft.